Lesson 3: JavaScript Objects

Intro to Programming — Code the Dream

Session Overview

By the end of this session, you will be able to:

- Describe what an object is and why we use one
- Create an object with properties and access its values
- Modify, add, and delete object properties
- Write a method inside an object using `this`
- Loop through object properties with `for...in`
- Build and use an array of objects
- Create objects from a constructor function
- Commit and push changes to GitHub

Explore

What Is an Object?

A **primitive** stores one value. An **object** groups many values together.

```
// Primitive – one value
let petName = "Teddy";

// Object – many related values grouped together
let myPet = {
  name: "Teddy",
  species: "ferret",
  color: "brown"
};
```

Think of an object like a profile card — each piece of info has a label.

Keys and Values

Every item inside an object has two parts: a **key** and a **value**.

```
let myPet = {
  name: "Teddy",     // key: "name",    value: "Teddy"
  species: "ferret" // key: "species", value: "ferret"
};
```

- The **key** is the label (always a string)
- The **value** can be any data type — string, number, boolean, even a function

> Separate key-value pairs with a comma. No comma after the last one (though JavaScript is forgiving).

Accessing Properties

Two ways to read a property from an object:

Dot notation — most common:

```
console.log(myPet.name);      // "Teddy"
console.log(myPet.species);   // "ferret"
```

Bracket notation — useful when the key is stored in a variable:

```
let key = "color";
console.log(myPet[key]);      // "brown"
```

Modifying and Deleting Properties

Update an existing property:

```
myPet.name = "Henry";
console.log(myPet.name);  // "Henry"
```

Add a new property:

```
myPet.age = 3;
```

Delete a property:

```
delete myPet.color;
console.log(myPet.hasOwnProperty("color")); // false
```

Check for Understanding

What does this code print?

```
const user = { name: "Alex", score: 10 };
user.score = 25;
console.log(user.score);
```

A) `10` — the original value

B) `25` — the updated value

C) `undefined` — `score` was deleted

D) An error — you cannot change a `const`

Looping Through an Object

Use a `for...in` loop to visit every property:

```
for (let key in myPet) {
  console.log(key + ":", myPet[key]);
}
// name: Henry
// species: ferret
// age: 3
```

Inside the loop, `key` is a variable — so you must use **bracket notation**: `myPet[key]`.

Object Methods

A **method** is a function stored as a property of an object.

```
let myPet = {
  name: "Teddy",
  species: "ferret",
  describe: function() {
    return `${this.name} is a ${this.species}.`;
  }
};

console.log(myPet.describe());
// "Teddy is a ferret."
```

`this` refers to **the object the method belongs to**.

Check for Understanding

What does `this` refer to inside a method?

```
let car = {
  brand: "Toyota",
  describe: function() {
    return "This car is a " + this.brand;
  }
};
```

A) The `describe` function itself

B) The `car` object

C) The global window object

D) The word "this" — it is just a string

Arrays of Objects

You can put objects inside an array. This is very common.

```
let pets = [
  { name: "Teddy", species: "ferret", color: "brown" },
  { name: "Oshie", species: "cat",    color: "orange" },
  { name: "Sunny", species: "dog",    color: "black" }
];
```

Loop through them with `forEach`:

```
pets.forEach(function(pet) {
  console.log(pet.name, "the", pet.species);
});
```

Constructor Functions

A constructor function is a **blueprint** for making many similar objects.

```
function Dog(name, breed, age) {
  this.name  = name;
  this.breed = breed;
  this.age   = age;
}

let dog1 = new Dog("Kroger", "greyhound", 8);
let dog2 = new Dog("Destiny", "shepherd", 14);
console.log(dog1);
// Dog { name: 'Kroger', breed: 'greyhound', age: 8 }
```

> The `new` keyword creates a fresh object each time.

Check for Understanding

What does the `new` keyword do?

```
let dog1 = new Dog("Rex", "labrador", 4);
```

A) Runs the function and prints the result

B) Creates a new object using `Dog` as a blueprint

C) Creates a copy of an existing `Dog` object

D) Deletes the previous `Dog` object

The Built-In Date Object

JavaScript has built-in objects. `Date` is one of them.

```
let currentDate = new Date();
console.log(currentDate);
// 2025-09-13T23:47:23.858Z

let year  = currentDate.getFullYear(); // 2025
let month = currentDate.getMonth();    // 8 (zero-indexed!)
let day   = currentDate.getDate();     // 13
```

> Months are **zero-indexed**: January = 0, December = 11.

Apply

Core Activity: Let's Code Together

We'll build a pet profile step by step — **you tell me what to type.**

Open your browser console:

- **Chrome / Edge:** Right-click anywhere → Inspect → Console
- **Firefox:** Right-click anywhere → Inspect → Console

This mirrors what you'll do in the assignment.

Step 1 — Create the Object

Ask the group: What three properties should our pet have?

```
let myPet = {
  name: "Mochi",
  species: "rabbit",
  color: "white"
};

console.log("Q1 object:", myPet);
console.log("Q1 name:", myPet.name);
```

Follow-up: How do we read just the species?

Step 2 — Modify a Property

Ask the group: How do we change the pet's name?

```
myPet.name = "Coco";
console.log("Q2 updated object:", myPet);
```

Ask the group: What will the output look like now?

Step 3 — Loop Through Properties

Ask the group: How do we print every property automatically?

```
for (let key in myPet) {
  console.log("Q3: " + key + ":", myPet[key]);
}
```

Ask the group: Why do we write `myPet[key]` instead of `myPet.key`?

Step 4 — Add a Method

Challenge: Add a `describe` method that returns a full sentence.

```
myPet.describe = function() {
  return `${this.name} is a ${this.color} ${this.species}.`;
};

console.log("Q4:", myPet.describe());
```

Ask the group: What does `this.name` refer to here?

Step 5 — Delete a Property

Challenge: Remove the `color` property and verify it is gone.

```
delete myPet.color;
console.log("Q5 Color deleted:", !myPet.hasOwnProperty("color"));
```

Ask the group: What do you expect `hasOwnProperty` to return after deleting?

Step 6 — Array of Objects + forEach

Challenge: Build a pets array and print each one.

```
let pets = [
  { name: "Mochi",  species: "rabbit", color: "white" },
  { name: "Shadow", species: "cat",    color: "black" },
  { name: "Pebble", species: "dog",    color: "tan"   }
];

function printPets(arr) {
  arr.forEach(function(pet) {
    console.log(pet);
  });
}
printPets(pets);
```

Extension Activity

Only if time allows and students are ready.

Extension — Compare Two Objects

Challenge: Write a function that checks whether two objects have the same keys and values.

```
function areObjectsEqual(obj1, obj2) {
  let keys1 = Object.keys(obj1);
  let keys2 = Object.keys(obj2);
  if (keys1.length !== keys2.length) return false;
  for (let key of keys1) {
    if (obj1[key] !== obj2[key]) return false;
  }
  return true;
}
```

This is Q8 in the assignment — the hardest question.

Assignment Preview

10 questions this week — all about objects:

Questions	Topic
Q1–Q2	Create an object and modify a property
Q3	Loop through properties with `for...in`
Q4	Add a `describe` method (use `this` for the stretch goal)
Q5	Delete a property and verify with `hasOwnProperty`
Q6	Array of objects with `forEach`
Q7	Constructor function — create two `Dog` instances
Q8	Compare two objects for equal keys and values
Q9–Q10	Built-in `Date` object — get year, month, and day

Git This Week: Commit and Push

This week you save your work to GitHub for the first time.

```
git checkout -b lesson-3
git add .
git commit -m "add index.html and update README with my name"
git push origin lesson-3
```

- `checkout -b` — creates a new branch
- `add .` — stages all your changes
- `commit -m` — saves a snapshot with a message
- `push` — sends it to GitHub

> Paste your `lesson-3` branch link in the "second link" field when you submit.

Tips for the Assignment

1. **Read the comment above each question** before writing any code
2. **Use `console.log()` on everything** — don't guess, verify
3. **Work through Q1–Q6 first** — they connect directly to what we just coded
4. **Stuck on Q8?** Ask an AI for hints, not the answer:

> "My areObjectsEqual function returns the wrong result for objects with different numbers of keys. Give me 3 hints without showing me the solution."

Wrap-Up

Zoom chat: Rate your confidence from 1 to 5

> How comfortable do you feel creating and working with objects?

Before next session:

- Read the **lesson materials** (JavaScript.info Objects, MDN Object Basics)
- Complete the **10-question assignment**
- **Commit and push** your work to your `lesson-3` branch on GitHub
- Post questions in **Slack** anytime — we check it between sessions

Resources

Resource	What it covers
JavaScript.info — Objects	Full written explanation
MDN — Object Basics	Reference and examples
W3Schools — for...in Loop	Loop reference (Q3)
W3Schools — JavaScript Dates	Date object reference (Q9–Q10)
Slack `#discussion` channel	Questions between sessions

> **Next week:** JavaScript Algorithms!

Great work today!

Objects help you organize your data like a pro.

Questions? Reach out on Slack anytime.